Name:R.Poornima siva priya
Reg:192425181

C0-1 AT-3

EXPERIMENTS

Experiment 1: Implementation of a Single Artificial Neuron Objective: Implement a single artificial neuron using Python

Task:

Develop a Python program to simulate an artificial neuron that accepts multiple inputs, computes the weighted sum, applies a step activation function, and generates the output. Test the neuron using different input combinations and analyze its behavior.

```
import numpy as np
def step_function(x):
    if x >= 0:
        return 1
    else:
        return 0
inputs = [
    [0,0],
    [0,1],
    [1,0],
    [1,1]
]
weights = [1,1]
bias = -1
print("Single Artificial Neuron")
print("--------------------------")
for x in inputs:
    weighted_sum = np.dot(x, weights) + bias
    output = step_function(weighted_sum)
    print("Input:", x,
          " Weighted Sum:", weighted_sum,
          " Output:", output)
```

Out put:

```
Single Artificial Neuron
-------------------------
Input: [0, 0]  Weighted Sum: -1  Output: 0
Input: [0, 1]  Weighted Sum: 0  Output: 1
Input: [1, 0]  Weighted Sum: 0  Output: 1
Input: [1, 1]  Weighted Sum: 1  Output: 1
```

Experiment 2: Perceptron for AND Gate Objective: Implement the Perceptron Learning Algorithm.
Task:

Write a Python program to implement a perceptron for the AND logic gate. Train the model using suitable weights and bias, display the final weights after training, and evaluate the prediction accuracy.

Code:

```
import numpy as np

X = np.array([

   [0,0],

   [0,1],

   [1,0],

   [1,1]

])

y = np.array([0,0,0,1])

weights = np.zeros(2)

bias = 0

learning_rate = 0.1

epochs = 10

def activation(x):

   return 1 if x >= 0 else 0

for epoch in range(epochs):

   for i in range(len(X)):

      net = np.dot(X[i], weights) + bias

      output = activation(net)

      error = y[i] - output

      weights += learning_rate * error * X[i]

      bias += learning_rate * error
```

```
print("Final Weights:", weights)

print("Final Bias:", bias)

correct = 0

print("\nPredictions")

for i in range(len(X)):

   net = np.dot(X[i], weights) + bias

   pred = activation(net)

   print(X[i], "->", pred)

   if pred == y[i]:

      correct += 1

accuracy = (correct/len(X))*100

print("\nAccuracy =", accuracy,"%")
```

Output:

```
Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004

Predictions
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1

Accuracy = 100.0 %
```

Experiment 3: Perceptron for OR Gate Objective: Design a perceptron model for binary classification

Task: Implement a perceptron to classify the OR logic gate. Compare the training process and final weights with those obtained for the AND gate and analyze the differences.

Code:

```
import numpy as np

X = np.array([

   [0,0],

   [0,1],

   [1,0],

   [1,1]
```

```
])
y = np.array([0,1,1,1])
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10
def activation(x):
    return 1 if x >= 0 else 0
for epoch in range(epochs):
    for i in range(len(X)):
        net = np.dot(X[i],weights)+bias
        output = activation(net)
        error = y[i]-output
        weights += learning_rate*error*X[i]
        bias += learning_rate*error
print("Final Weights =",weights)
print("Final Bias =",bias)
correct=0
print("\nPredictions")
for i in range(len(X)):
    net=np.dot(X[i],weights)+bias
    pred=activation(net)
    print(X[i],"->",pred)
    if pred==y[i]:
        correct+=1
accuracy=(correct/len(X))*100
print("\nAccuracy =",accuracy,"%")
```

Output:

```
Final Weights = [0.1 0.1]
Final Bias = -0.1

Predictions
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 1

Accuracy = 100.0 %
```

Experiment 4: XOR Classification Analysis Objective: Study the limitations of a single-layer perceptron.

Task: Attempt to train a perceptron for the XOR logic gate. Observe the results and explain why the perceptron fails to classify XOR correctly. Suggest an alternative neural network architecture capable of solving this problem.

Code:

```
import numpy as np
X=np.array([
  [0,0],
  [0,1],
  [1,0],
  [1,1]
])
y=np.array([0,1,1,0])
weights=np.zeros(2)
bias=0
learning_rate=0.1
epochs=20
def activation(x):
  return 1 if x>=0 else 0
for epoch in range(epochs):
  for i in range(len(X)):
    net=np.dot(X[i],weights)+bias
    output=activation(net)
    error=y[i]-output
```

```
        weights+=learning_rate*error*X[i]
        bias+=learning_rate*error
print("Final Weights =",weights)
print("Final Bias =",bias)
print("\nPredictions")
correct=0
for i in range(len(X)):
    net=np.dot(X[i],weights)+bias
    pred=activation(net)
    print(X[i],"->",pred)
    if pred==y[i]:
        correct+=1
accuracy=(correct/len(X))*100
print("\nAccuracy =",accuracy,"%")
print("\nObservation")
print("A single-layer perceptron cannot correctly classify XOR because XOR is not linearly separable.")
print("A Multi-Layer Perceptron (MLP) with one hidden layer can solve the XOR problem.")
```

Output:

```
Final Weights = [-0.1  0. ]
Final Bias = 0.0

Predictions
[0 0] -> 1
[0 1] -> 1
[1 0] -> 0
[1 1] -> 0

Accuracy = 50.0 %

Observation
A single-layer perceptron cannot correctly classify XOR because XOR is not linearly separable.
A Multi-Layer Perceptron (MLP) with one hidden layer can solve the XOR problem.
```

Experiment 5: Gradient Descent Optimization Objective: Understand Gradient Descent.

Task: Implement the Gradient Descent algorithm to minimize a simple quadratic cost function. Plot the cost function against the number of iterations and analyze the convergence behavior for different learning rates.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
def cost(x):
    return x**2
def gradient(x):
    return 2*x
x = 8
learning_rate = 0.1
iterations = 50
cost_history = []
x_history = []
for i in range(iterations):
    x = x - learning_rate*gradient(x)
    cost_history.append(cost(x))
    x_history.append(x)
print("Final x =",x)
print("Final Cost =",cost(x))
plt.figure(figsize=(8,5))
plt.plot(cost_history,marker='o')
plt.title("Gradient Descent Convergence")
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.grid(True)
plt.show()
```

Output: